

Instituto Tecnológico De Costa Rica

Curso: IC6600

Principios de sistemas operativos

PY #1

Prof. [Cristian Paz Campos Agüero](#)

Est. Poll Garro Vargas

Grupo 60

II Semestre

2026

1. Introducción

El proyecto simula una minicomputadora compuesta por una CPU, memoria principal, almacenamiento secundario (disco), pantalla y teclado. Sobre ella funciona un pequeño sistema operativo que administra la ejecución de varios programas escritos en un mini ensamblador (archivos .asm).

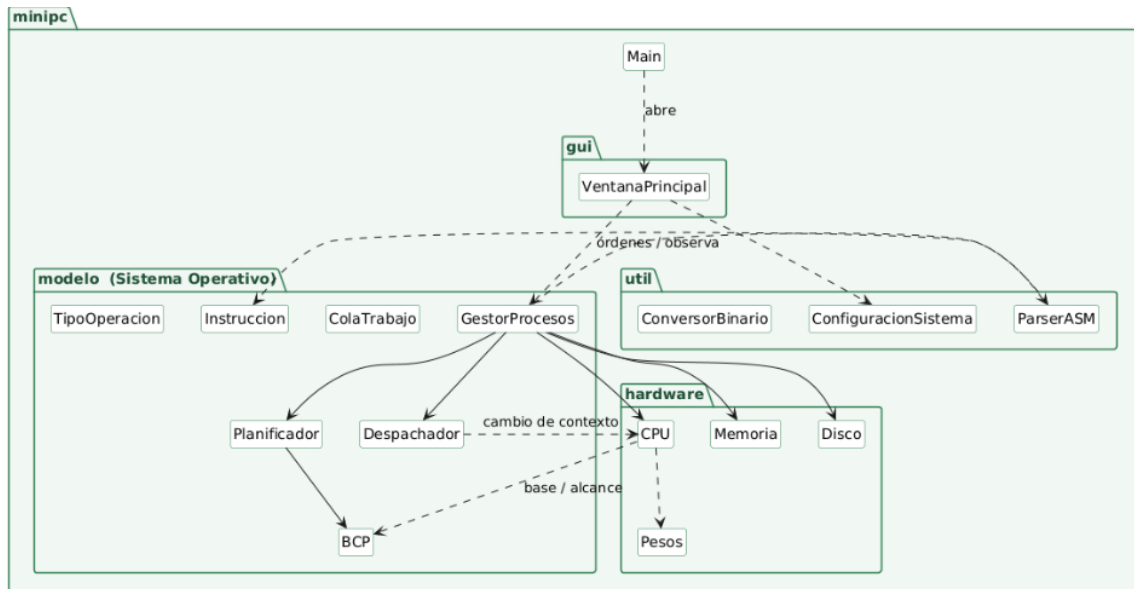
El sistema operativo carga uno o varios programas y los guarda en disco. Por cada uno crea un proceso con su Bloque de Control de Proceso (BCP), lo pasa a memoria cuando hay espacio disponible y lo ejecuta mediante planificación FCFS First Come, First Served. La interfaz gráfica permite observar en todo momento el estado de la CPU, la memoria, el disco, los BCP y la lista de trabajos.

El diseño sigue los conceptos presentados por William Stallings en Operating Systems: Internals and Design Principles (9.ª ed.):

1. Bloque de control de proceso (BCP)
2. Modelo de siete estados
3. Despachador y cambio de contexto
4. Planificación FCFS
5. Protección de memoria
6. Memoria virtual

2. Diagrama de paquetes

El código se divide en cuatro paquetes dentro de minipc. La separación responde a una idea simple: el hardware no conoce al sistema operativo (solo ejecuta instrucciones), el sistema operativo administra el hardware y la interfaz solo observa y le da órdenes al sistema operativo.



2.1 Responsabilidad de cada paquete

Paquete	Clases	Responsabilidad
---------	--------	-----------------

hardware	CPU, Memoria, Disco, Pesos	La máquina simulada. La CPU ejecuta el ciclo <i>fetch</i> → <i>decode</i> → <i>execute</i> ; la memoria y el disco son arreglos de posiciones; Pesos indica cuántos segundos tarda cada instrucción.
modelo	GestorProcesos, Planificador, Despachador, BCP, ColaTrabajo, Instruccion, TipoOperacion	El sistema operativo: creación de procesos, colas, planificación FCFS, despacho, cambio de contexto, memoria virtual y suspensión.
util	ParserASM, ConfiguracionSistema, ConversorBinario	Lectura y validación de los archivos .asm y de la configuración (config.txt).
gui	VentanaPrincipal	Interfaz gráfica. No contiene lógica del S.O.: muestra el estado y delega todas las acciones en GestorProcesos.

2.2 Clases principales

Clase	Función en el diseño
GestorProcesos	Punto de entrada del S.O. Crea procesos, avanza un segundo de CPU (ejecutarUnPaso), atiende el teclado, controla errores y genera estadísticas.
Planificador	Planificador de trabajos (largo plazo) y de CPU (corto plazo, FCFS). Mantiene la lista de trabajos, la de procesos, la de espera y la de finalizados. Decide las cargas a memoria, el uso de memoria virtual y las suspensiones.
Despachador	Pone un proceso en la CPU restaurando su contexto y guarda el contexto del proceso que sale.
BCP	Bloque de Control de Proceso: identificador, estado, PC, registros, pila de 5 posiciones, base, alcance, prioridad, CPU asignada, tiempos, archivos abiertos y enlace al siguiente BCP.
ColaTrabajo	Cola FIFO enlazada de BCP, donde cada BCP apunta al siguiente.
CPU	Registros PC, IR, AC, AX, BX, CX, DX, AH, AL y bandera de comparación. Ejecuta las 21 instrucciones y las interrupciones.
Memoria / Disco	Memoria principal: zona del S.O. y zona de usuario, búsqueda de huecos y validación de acceso. Disco: índice, área de archivos y memoria virtual.
ParserASM	Valida sintaxis, operandos, registros, rangos e interrupciones; en cada error informa el archivo, la línea y el motivo.

3. Arquitectura y flujo de un proceso

Un programa recorre las siguientes etapas desde que se carga hasta que termina. La interfaz solo llama a GestorProcesos, y este coordina al planificador, al despachador y al hardware.

1. **Carga y validación.** El usuario selecciona uno o varios archivos .asm. ParserASM revisa cada línea; si encuentra un error, el archivo se rechaza indicando la línea y el motivo.
2. **Almacenamiento en disco.** El programa válido se guarda en el área de archivos del disco y se registra en el índice.
3. **Creación del proceso.** Se crea su BCP en estado **Nuevo** y se agrega a la **lista de trabajos**.
4. **Admisión a memoria.** El planificador de trabajos lo pasa a memoria si hay menos de 5 procesos cargados y existe un hueco:
 - Si cabe completo en la zona de usuario, se carga ahí.
 - Si no cabe completo, una parte se carga en memoria y el resto en la **memoria virtual** del disco.
 - Si es más grande que la memoria de usuario más la virtual, se rechaza por **desbordamiento de memoria**.
 - Si ya hay 5 procesos, espera en la lista de trabajos.

Al quedar cargado, su BCP se guarda en la zona del S.O. y pasa a **Preparado**.

5. **Planificación FCFS.** Cuando la CPU queda libre, el planificador elige el primer proceso preparado según el orden de llegada.
6. **Despacho.** El despachador restaura el contexto del proceso en la CPU (PC, registros, pila) y lo pone en **Ejecución**.
7. **Ejecución.** Cada segundo la CPU avanza un *tick*; una instrucción se completa cuando se cumple su peso. Con INT 09H, el proceso queda **En espera** dentro de la CPU hasta que el usuario ingrese un valor, y su tiempo sigue contando.
8. **Finalización.** Con INT 20H, o si ocurre un error de ejecución, el proceso pasa a **Finalizado**: se registra su hora de fin, se libera su memoria y su espacio de BCP, y entra el siguiente de la lista de trabajos.



5. Planificación, despachador y cambio de contexto

5.1 Planificador de trabajos (largo plazo)

Planificador.prepararSiguientes() toma, en orden de llegada, los programas de la lista de trabajos y los pasa a memoria mientras haya espacio y haya menos de 5 procesos en memoria (contando el que está en la CPU y los que esperan). Si el primero de la lista no puede entrar, los demás también esperan, para respetar FCFS estricto.

5.2 Planificador de CPU (corto plazo): FCFS

Planificador.siguienteProceso() entrega el proceso que lleva más tiempo en la lista de procesos. La lista es una cola FIFO (ColaTrabajo) enlazada por los propios BCP, donde cada uno apunta al siguiente. La planificación es **no expropiativa**: un proceso conserva la CPU hasta que termina.

5.3 Despachador y cambio de contexto

El Despachador realiza el cambio de contexto mediante dos operaciones:

- **capturar(bcp)**: copia del CPU al BCP del proceso que sale el PC, IR, AC, AX, BX, CX, DX, AH, AL y la bandera de comparación.
- **despachar(bcp)**: carga en la CPU los registros guardados del proceso que entra y lo marca en **Ejecución**.

El cambio de contexto ocurre cuando el proceso termina (INT 20H) o cuando produce un error. Con INT 09H no hay cambio de contexto: el proceso conserva la CPU mientras espera el número. La pila de cada proceso vive en su propio BCP, por lo que no se mezcla entre procesos.

5.4 Tiempo y pesos

Cada llamada a GestorProcesos.ejecutarUnPaso() equivale a 1 segundo de CPU. La CPU hace *fetch* y *decode* en el primer segundo y ejecuta la instrucción cuando se cumplen los segundos de su peso (por ejemplo, ADD tarda 3 s).

- **Siguiente**: avanza un segundo por cada clic.
- **Ejecutar**: con la velocidad **1 segundo**, avanza un segundo de CPU por cada segundo real; con **Instantánea**, ejecuta todo de una vez.

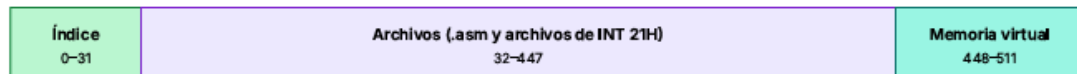
Mientras un proceso espera un valor de teclado (INT 09H), el reloj sigue avanzando y ese tiempo se suma a su tiempo empleado.

6. Gestión de memoria

Memoria principal (256)



Disco (512)



6.1 Memoria principal

La memoria principal se divide en dos zonas:

Zona	Tamaño	Contenido
Sistema operativo	25 % de la memoria	Los BCP de los procesos cargados

6.2 Memoria virtual

La memoria virtual ocupa el último 12.5 % del disco.

- Si el programa cabe completo en la RAM, se carga completo.
- Si no cabe, se carga en el hueco más grande todo lo que quepa y el resto se escribe en la memoria virtual del disco.
- Cuando el PC llega a esa parte, la CPU lee la instrucción directamente desde la memoria virtual. Un S.O. real la traería a la RAM mediante un fallo de página; aquí se simplificó leyéndola directamente.
- Si no hay espacio ni en la RAM ni en la memoria virtual, el proceso espera en la lista de trabajos.
- Si el programa es más grande que la RAM de usuario más la memoria virtual, se rechaza por **desbordamiento de memoria**.

6.3 Suspensión

El planificador permite suspender un proceso bloqueado fuera de la CPU cuando el siguiente trabajo no puede entrar a memoria. Al suspenderlo, libera su RAM, su memoria virtual y su BCP, y guarda la instrucción en la que iba. Cuando vuelve, se recarga desde el disco y su PC se reubica según la nueva base.

Con la regla de INT 09H usada en este proyecto (el proceso espera dentro de la CPU), este caso no llega a ocurrir.

7. Interrupciones, llamadas al sistema y almacenamiento

Interrupción	Comportamiento
--------------	----------------

INT 20H	Finaliza el programa: el proceso pasa a Finalizado y libera sus recursos.
INT 10H	Imprime el contenido de DX en la pantalla (número o texto), con el prefijo del proceso, por ejemplo P2 > 15.
INT 09H	Solicita un número de 0 a 255. El proceso pasa a En espera sin salir de la CPU y el reloj sigue contando. Al recibir el valor, este se guarda en DX y el proceso continúa con la siguiente instrucción.
INT 21H	Manejo de archivos en el disco simulado. DX contiene el nombre del archivo, AH la función y AL el dato leído o escrito. Los archivos aparecen en el índice del disco y en la lista de archivos abiertos del BCP.

El disco mantiene en sus primeros registros un índice con el nombre, la dirección y el tamaño de cada archivo. Los programas cargados se guardan ahí antes de crear el proceso, tal como lo pide el enunciado.

8. Protección y seguridad (estrategia)

La estrategia combina protección por hardware simulado, aislamiento de procesos y validación previa:

- **Base y límite.** En cada *fetch*, la CPU valida que el PC esté dentro de [base, base + alcance) del proceso (Memoria.validarAcceso). Un proceso no puede leer código de otro ni de la zona del S.O.
- **Salto controlados.** JMP, JE y JNE verifican que el destino esté dentro del proceso; si no, se produce un error de desbordamiento.
- **Pila protegida.** La pila (5 posiciones) detecta el desbordamiento y la pila vacía. PARAM verifica que quepan todos los valores antes de insertar alguno.
- **Aislamiento de errores.** Cualquier error en ejecución termina únicamente al proceso que lo causó; los demás siguen ejecutándose.
- **Validación previa.** ParserASM rechaza antes de la carga los programas con operaciones, registros, operandos, valores o interrupciones inválidos.
- **Separación S.O. / usuario.** Los programas solo se cargan en la zona de usuario; los BCP viven en la zona del S.O.
- **Archivos protegidos.** INT 21H no puede crear, modificar ni eliminar archivos .asm.
- **Configuración validada.** Los tamaños leídos de config.txt se validan antes de usarse.

9. Configuración e interfaz

Los tamaños de la memoria principal y del disco se leen de config.txt (ConfiguracionSistema), por lo que no quedan fijos en el código. La zona del S.O. se calcula como el 25 % de la memoria principal y la memoria virtual como el 12,5

% del disco. Estos valores también pueden cambiarse desde el botón

Configuración. Al cerrar la aplicación, el archivo vuelve a los valores por defecto:

Parámetro	Valor por defecto
Memoria principal	256
Zona del S.O. (25 %)	64
Disco	512
Memoria virtual (12,5 %)	64

La interfaz muestra

- la cola de trabajos con sus estados;
- los recursos del sistema y los dispositivos;
- la memoria principal (mapa y tabla) y el disco;
- la pantalla con entrada de teclado;
- los registros de la CPU, la pila y el BCP actual.

Al terminar, el botón **Estadísticas** muestra para cada proceso la hora y minuto de inicio y de fin, la duración en segundos y el tiempo de CPU.

Decisiones de diseño

1. Se usa el modelo de 7 estados de Stallings.
2. El peso de INT 09H es 1 s. El proceso espera el valor dentro de la CPU sin cambio de contexto, y el reloj sigue contando hasta recibirlo.
3. Los procesos que exceden el límite de 5 esperan en la lista de trabajos. La memoria virtual se reserva para programas que no caben completos en la RAM.